TRAINING REPORT

ON

E-Commerce Web App

MASTER OF COMPUTER APPLICATIONS

in

DEPARTMENT OF COMPUTER APPLICATION

Engineering College Bikaner

Session-2024-2026

Submitted By: Gagan Saini

CDTS
Cloud & DevOps

Certificate of Completion

This is to certify that Mr. Gagan Saini S/o Sh. Anand Prakash Saini student of MCA from Engineering College Bikaner, Bikaner has successfully completed an industrial training cum internship program on **"MERN Stack"** and also worked on the same project.

The duration of internship was 45 days from 19.01.2026 to 05.03.2026

During the internship session his behaviour was good.

NARENDRA SINGH BHUI
Academy Director

Certificate number: CDTS/CERT/Gen-File/251
Issued date: 05.03.2026

URL: https://www.cdtsbikaner.in Contact: +91-7742163889
Behind Rajasthan Patrika Press, Amar Singh Pura, Bikaner PIN 334001 (Rajasthan) INDIA
Society Reg. No. COOP/2020/BIKANER/200308

Society Reg No: COOP/2020/BIKANER/200308

CDTS
Cloud & DevOps

Centre for Cloud & DevOps Training Society, Bikaner

(A Complete IT Solutions for Open-Source Training and Software Development)
Amar Singh Pura, Behind Rajasthan Patrika Press, Bikaner-334001. Contact: +91-7742163889
Website: https://www.cdtsbikaner.in , Email: info@cdtsbikaner.com , cdtsbikaner@gmail.com

F.No. /CDTS/ 2026/MCA -Internship/151

Date: 05-03-2026

TO WHOM IT MAY CONCERN

This is to certify that Mr. Gagan Saini S/o Sh. Anand Prakash Saini, student of MCA from Engineering College Bikaner, Bikaner (Rajasthan) has successfully completed an industrial training cum internship program on **"MERN Stack"** and also worked on the same projects (e-Commerce solutions). The duration of the internship was 45 days between 19.01.2026 to 05-03-2026.

During the period of his training program with us found punctual, hardworking, and inquisitive.

We wish him success in life.

Narendra Singh Bhui
(Director)

C D T S
BIKANER

TABLE OF CONTENTS

1. Executive Technical Overview
2. Frontend Architecture
3. Backend Architecture
4. Authentication & Authorization
5. Database Design
6. Cart System Architecture
7. Checkout & Order Architecture
8. Payment Processing Architecture
9. Product Catalog & Search
10. Inventory Design

1. EXECUTIVE TECHNICAL OVERVIEW

Closet Fashion is a full-stack fashion e-commerce platform built for individual buyers and independent sellers. The platform enables product discovery, cart management, direct purchase, seller onboarding, and order history. It is deployed entirely on Vercel -- the frontend as a static React SPA and the backend as a serverless Node.js function set.

The system is architecturally a two-tier monolith: a React single-page application communicating directly over HTTP with a single Express.js server backed by a MongoDB Atlas cluster. There are no microservices, no message queues, no caching layers, and no background workers. All business logic lives inside controller functions co-located in a single Express process.

PLATFORM SCALE CONTEXT

This platform is at an early-stage, MVP product maturity level. The architectural decisions -- embedded cart/order arrays in user documents, base64 image storage in MongoDB, no payment processing, single-tier Express -- reflect a build-and-ship philosophy prioritizing velocity over

operational durability. Many production-grade concerns (rate limiting, observability, inventory control, real payment integration) are deliberately deferred or entirely absent.

This document does not judge that posture. It accurately describes what exists,why patterns were chosen, where the risk surface lies, and what must change at each growth threshold.

TECHNOLOGY STACK SUMMARY

The e-commerce platform follows a modern full-stack JavaScript architecture designed for scalability, maintainability, and rapid deployment. The frontend layer is built using React version 18.2.0, enabling a component-driven UI architecture with efficient rendering and state-driven updates. The frontend application is deployed through the Vercel CDN infrastructure, allowing globally distributed static asset delivery with low-latency performance optimization. Application bundling and production build generation are handled through Create React App using `react-scripts` version 5.0.1, providing a standardized build pipeline, environment management, asset optimization, and production-ready static compilation.

Client-side routing is implemented using React Router DOM version 6.7.0, enabling single-page application navigation without full page reloads. This routing architecture improves user experience through faster transitions and reduced server dependency during navigation events. Styling and UI consistency are managed using Tailwind CSS version 3.4.4, integrated through a PostCSS-based build workflow. This utility-first CSS architecture improves maintainability, reduces custom stylesheet complexity, and enables responsive design implementation with minimal overhead. Communication between the frontend and backend services is performed using Axios version 1.2.4, operating within the browser environment to manage asynchronous HTTP requests, API integration, request interception, and centralized error handling workflows.

The backend infrastructure is built on Node.js with Express version 4.18.2 serving as the primary application framework. The backend is deployed using Vercel Serverless Functions, enabling automatic scaling, simplified deployment pipelines, and reduced infrastructure management overhead. The application database layer uses MongoDB Atlas as the managed cloud database provider, while Mongoose version 6.8.3 acts as the Object Data Modeling (ODM) layer responsible for schema validation, model abstraction, query management, and middleware integration. The database is hosted on the MongoDB Atlas M0 shared cluster tier, which provides a lightweight managed database environment suitable for small-to-medium scale workloads and development-stage deployments.

Authentication within the platform is implemented using JSON Web Tokens through the `jsonwebtoken` package version 9.0.0. The system follows a stateless authentication architecture where access tokens are generated and verified without requiring server-side session persistence. This approach improves horizontal scalability and simplifies distributed deployment handling. Password security is implemented using bcrypt version 5.1.0 for server-side password hashing and credential verification. The hashing mechanism ensures sensitive user credentials are never stored in plaintext and provides resistance against common credential compromise attacks.

For user interaction feedback and alert management, the application integrates SweetAlert version 2.1.2 within the browser environment. This library is used to provide modal-based notifications, confirmation dialogs, validation feedback, and transactional alerts across critical user workflows such as authentication, checkout actions, order placement, and administrative operations.

LIVE DEPLOYMENT ENDPOINTS

Frontend: https://closet-fashion.vercel.app
Backend API: https://cf-backend-ecru.vercel.app
Database: MongoDB Atlas cluster0.vpff1ir.mongodb.net

2. FRONTEND ARCHITECTURE

RENDERING MODEL

Closet Fashion uses Client-Side Rendering (CSR) exclusively, bootstrapped via Create React App (CRA). The public/index.html ships a bare `<div id="root"></div>` shell. React hydrates it on the client.

Why CSR only: CRA was chosen for its zero-configuration development setup. There is no Next.js or Remix SSR layer. This means:

1. First Contentful Paint (FCP) is delayed -- the browser must download the JS bundle, parse it, execute React, then fetch data from the API before anything meaningful renders.

2. SEO is non-existent -- product pages are not server-rendered, so web crawlers see empty HTML. Product catalog is invisible to search engines.

3. No static generation -- product listing pages are always fetched at runtime.

The tradeoff accepted here is simplicity. CRA produces a fully static artifact (HTML + JS + CSS) that Vercel can serve from CDN globally with zero server infrastructure for the frontend layer.

APPLICATION ENTRY POINT

```
src/index.js
  +-- ReactDOM.createRoot(document.getElementById('root'))
      +-- <App />
          +-- createBrowserRouter([...routes])
              +-- RouterProvider
```

index.js is intentionally thin -- it creates the React root and calls reportWebVitals() (which logs to console but sends metrics nowhere). The router and context providers are configured inside App.jsx.

ROUTING ARCHITECTURE

React Router DOM v6 with createBrowserRouter and the Outlet nested route pattern:

```
/                       -> <AppLayout />       (persistent navbar + outlet)
/login                  -> <Login />           (standalone, no layout)
/home                   -> <Home />            (product listing)
/product/:id            -> <ProductPage />     (product detail)
/product/buynow/:id -> <BuyNow />          (checkout flow)
/                       -> redirect -> /home
*                       -> redirect -> /home
```

AppLayout is the persistent shell component. It renders <Navbar>, wraps everything in <SearchContextProvider>, and renders <Outlet> for child routes.

The layout is always mounted when the user is on any route except /login.

The routing decision to keep /login outside AppLayout is correct -- the login page needs a completely different visual layout (split-screen design) with no navbar or filter panel.

Deep-link behavior: createBrowserRouter produces history-mode URLs. Vercel handles the SPA routing correctly because its build output config rewrites all paths to index.html. No 404s on direct URL access.

COMPONENT HIERARCHY

```
App
+-- ThemeProvider (context)
```

```
+-- Login (standalone route)
+-- AppLayout
   +-- SearchContextProvider (context)
   +-- Navbar
   |   +-- NavbarRightOptions
   |   +-- ProfileSlider (slide-over panel)
   |   |   +-- Profile tab -> Profile.jsx
   |   |   +-- Orders tab -> MyOrders.jsx
   |   |   +-- Seller tab -> SellerAccount.jsx OR UploadProduct.jsx
   |   |   +-- Settings tab -> Settings.jsx -> ManageTheme.jsx
   |   +-- CartSlider (slide-over panel)
   |       +-- Cart.jsx
   |           +-- CartCard.jsx (per item)
   |           +-- CheckoutCard.jsx (per item, right panel)
   +-- Outlet
      +-- Home
      |   +-- Filters
      |   |   +-- FilterCard (per filter group)
      |   |   +-- ColorFilter
      |   |   +-- RangePicker (price range)
      |   +-- ProductCard1 (per product)
      +-- ProductPage
      |   +-- ProductImages
      |   +-- ColorFilterCard (readonly)
      |   +-- Checkbox (readonly for sizes)
      +-- BuyNow
          +-- ProductImages
          +-- QuantityInput
          +-- Checkbox (size selection)
          +-- ColorFilterCard (color selection)
          +-- PaymentItem (per payment method)
```

STATE ARCHITECTURE

No global state manager (no Redux, no Zustand, no Jotai). State is organized in three tiers:

Tier 1 - React Context (global cross-cutting):
- ThemeContext -- light/dark mode, persisted to localStorage["cf-theme"]
- SearchContext -- current search query string, shared between Navbar search input and Home component

Tier 2 - App-level state (prop-drilled):
- authToken -- stored in App state, initialized from localStorage["authToken"], prop-drilled four levels deep to Navbar, ProfileSlider, Home, ProductPage, CartSlider
- showFilterSection -- boolean, controls filter panel visibility on mobile
- showCartSlider, showProfileSlider -- boolean toggles for slide-over panels

Tier 3 - Component-local state:
- allProducts, page, totalPages -- pagination state in Home
- selectedFilters -- multi-dimensional filter state in Home
- cartProducts, productPriceDetails -- cart data in Cart.jsx
- product, sellerDetails -- product detail data in ProductPage
- Form state is managed entirely by the useForm custom hook

Architectural tension: authToken is simultaneously stored in App component state and localStorage. Two reads occur -- once during render and once inside useEffect. Components that need the token independently read from localStorage (e.g., Home.jsx reads it directly via localStorage.getItem("authToken")), bypassing the prop-drilled state. This creates an inconsistency where the React state and localStorage can briefly diverge on logout. The fix would be a dedicated AuthContext.

CUSTOM HOOKS

1. useAPI (src/hooks/useAPI.js)

A generic data-fetching hook wrapping Axios:

useAPI(method, path, includeAuthToken, variables, headers, dependencies) { data, loading, error }

Fires on mount and whenever dependencies change. Uses useCallback to memoize fetchData with JSON.stringify(variables) and JSON.stringify(headers) as dependencies -- a common React pattern to avoid deep-equality comparisons, but one that causes unnecessary refetches when object references change even if values are identical.

2. useDebouncedAPI (src/hooks/useDebounceAPI.js)

A debounced variant of useAPI using useRef for the timeout handle. Default debounce delay is 200ms;
Home uses 250ms. Used for:
- Product listing (triggers on page change and filter change)
- Any search-triggered API calls

The debounce implementation correctly uses a ref rather than state to store the timer ID, avoiding the stale-closure issue present in the naive useDebounce hook.

3. useDebounce (src/hooks/useDebounce.js)

A lower-level debounce hook that stores the timer ID in useState rather than useRef. This is a bug: setting timerId via setTimerId causes a re-render, which re-triggers the useEffect, creating potential for a cascade. This hook is not used in the main product listing but exists as an earlier implementation. The useDebouncedAPI hook supersedes it.

4. useForm (src/hooks/useForm.js)

A generic controlled-form hook:
useForm(initialValues, validationRules, onSubmit)
-> { formData, errors, isSubmitting, setFormData, setErrors, handleChange, handleSubmit }

Validation rules are a map of fieldName -> (value, formData) -> errorMessage or undefined. Cross-field validation (e.g., card number required only when payment method is card) is supported via the second formData argument passed to each rule.

handleSubmit runs all validators synchronously, collects errors, and calls onSubmit only if validation passes. isSubmitting is set to true then immediately back to false -- it wraps no async operation directly, so it cannot be used as an async loading indicator.

THEME SYSTEM
Two themes: light and dark. Persisted to localStorage["cf-theme"], defaulting to light. Applied via conditional Tailwind class strings: className={theme === "light" ? "bg-White text-Black" : "bg-Black text-White"}

The custom Tailwind palette is:
- White: #F0F3F6 (cool off-white)
- Light: #DFE7F1 (light blue-gray)
- Black: #1E1E1E (near-black)
- Purple: #7B68EE (medium-slate-blue -- primary brand color)
- Gray: #999999
- Green: #48D198
- Red: #FF0000
- Blue: #0F4392
- Yellow: #EFC863

The theme toggle is buried inside the Profile Slider -> Settings tab. It is not accessible without being logged in (the slider is only shown when authToken is present). Theme toggle for guest users is unavailable.

INFINITE SCROLL IMPLEMENTATION

Home uses the Intersection Observer API for infinite scroll pagination:

```
  const lastProductElementRef = useCallback((node) => {
    if (loading) return;
    if (observer.current) observer.current.disconnect();
    observer.current = new IntersectionObserver((entries) => {
      if (entries[0].isIntersecting && page < totalPages) {
        setPage(prev => prev + 1);
      }
    });
    if (node) observer.current.observe(node);
  }, [loading, page, totalPages]);
```

This ref is attached to the last product card in the grid. When it enters the viewport, page increments, triggering a new API call which appends results to allProducts.

Critical behavior difference when filters are active: When any filter is applied, the backend ignores pagination and returns all matching results (offset is forced to 0). The frontend then replaces allProducts entirely rather than appending:

```
  if (filtersActive || searchQuery?.length > 0) {
    setAllProducts([...data?.products]);
  } else {
    setAllProducts(prev => [...prev, ...data?.products]);
  }
```

This means filtered searches return unbounded result sets -- a filter for "Nike" products returns every Nike product in one response, not paginated. For large catalogs this becomes a scalability problem.

3. BACKEND ARCHITECTURE

ARCHITECTURE PATTERN

The backend is a single-file Express monolith. All routes are registered in server.js. Controllers are organized into two directories:
- controllers/userControls/ -- 6 user management controllers
- controllers/productControls/ -- 11 product/cart/order controllers

There is no:
- Router-level modularization (express.Router())
- Service layer (business logic lives directly in controllers)
- Repository/DAO pattern (Mongoose models called directly in controllers)
- Dependency injection
- Middleware pipeline beyond CORS + body parsing + JWT auth

This is a classic fat controller pattern. Business logic, data access, and HTTP handling are all co-located in each controller file.

MODULE STRUCTURE

```
backend/
+-- server.js             <- Express app, all route registration
+-- config/
|   +-- db-config.js      <- MongoDB connection setup
+-- middleware/
|   +-- authenticateUser.js <- JWT verification
+-- models/
|   +-- Users.js          <- Mongoose User schema
|   +-- Products.js       <- Mongoose Product schema
+-- controllers/
|   +-- userControls/
|   |   +-- signup.js
|   |   +-- login.js
```

```
|   |   +-- user-details.js
|   |   +-- update-profile.js
|   |   +-- update-address.js
|   |   +-- update-phonenumber.js
|   |   +-- become-seller.js
|   +-- productControls/
|       +-- get-all-products.js
|       +-- get-product-by-id.js
|       +-- upload-product.js
|       +-- search-product.js     <- BROKEN
|       +-- add-to-cart.js
|       +-- access-cart-items.js
|       +-- remove-item.js
|       +-- buy-product.js
|       +-- checkout-product.js
|       +-- get-ordered-products.js
|       +-- ask-product-question.js
|       +-- set-product-review.js
+-- utils/
    +-- categoryList.js    <- Static enum data
    +-- companyList.js     <- Static enum data
    +-- prices.js          <- Static enum data
```

Note on utils: The backend utility files (categoryList.js, companyList.js, prices.js) define static enumeration data but are never imported anywhere in the codebase. They are dead code -- present in the repository but unused.
Enumerations are maintained only in the frontend config.js. This creates a data consistency risk: backend and frontend enum lists are maintained separately and can diverge.

EXPRESS CONFIGURATION

```
app.use(cors({
```

```
  origin: ["https://closet-fashion.vercel.app", "http://localhost:3000"],
  credentials: true,
}));
app.use(express.json({ limit: "50mb" }));
app.use(express.urlencoded({
  limit: "50mb", extended: true, parameterLimit: 50000
}));
```

The 50MB body limit exists specifically to accommodate base64-encoded product images included inline in request bodies. A single product upload can transmit 10-40MB of base64 image data in one POST. On a serverless function (Vercel), this is particularly problematic as cold starts and memory limits apply.

CORS is configured with a hardcoded origin whitelist. credentials: true is set, but the frontend uses Authorization headers rather than cookies, so this flag is effectively unused.

SERVERLESS DEPLOYMENT CONSIDERATIONS

The backend runs as a Vercel Serverless Function via @vercel/node. The vercel.json maps all routes to server.js:

```
{
  "version": 2,
  "builds": [{ "src": "server.js", "use": "@vercel/node" }],
  "routes": [{ "src": "/(.*)", "dest": "/server.js" }]
}
```

Critical architectural implication: Vercel serverless functions are stateless and ephemeral. Each invocation may spin up a new function instance. The Express app and MongoDB connection are re-initialized per cold start. This has several consequences:

1. Connection pooling is unreliable. Mongoose maintains a connection pool designed for long-running processes. In a serverless environment, the connection is re-established on each cold start and may be terminated mid-pool by the platform. The connection string uses retryWrites=true which helps but does not solve the fundamental pool management issue.

2. No in-memory state. Nothing can be cached in application memory (no in-process Redis alternative, no in-memory session store). Every request that needs state must go to MongoDB.

3. Function timeout. Vercel serverless functions have a 10-second execution timeout on the Hobby plan. Large base64 image payloads being written to MongoDB can approach this limit.

4. app.listen(port, ...) is a no-op on Vercel. Vercel does not use the Express HTTP server directly; it wraps the Express app as a serverless handler. The app.listen(5000, ...) call in server.js is executed but the port binding is ignored by the Vercel runtime. This works correctly in practice but creates confusion when reading the code.

DATABASE CONNECTION MANAGEMENT

```
const connectDB = async () => {
  try {
    await mongoose.connect(DB_URL);
    console.log("Database Connected Successfully");
  } catch (error) {
    console.error("MongoDB connection error:", error);
    process.exit(1);
  }
};
```

connectDB() is called at module load time in server.js. On serverless, this means it runs on every cold start. The process.exit(1) on connection failure is correct for a long-running server process but problematic for serverless:

the function will simply fail rather than return a 503, leaving the caller with a connection timeout rather than a meaningful error response.

There is no connection lifecycle management: no mongoose.disconnect(), no health check endpoint, no reconnection logic beyond what Mongoose provides internally.

4. AUTHENTICATION & AUTHORIZATION

AUTHENTICATION FLOW

```
Signup / Login
     |
     v
bcrypt.hash(password, saltRounds=8)      <- password hashing
     |
     v
jwt.sign({ _id, name, email }, AUTH_TOKEN_SECRET)   <- no expiry set
     |
     v
res.json({ authToken })                  <- returned to client
     |
     v
localStorage.setItem("authToken", token) <- client storage
```

Token Structure: The JWT payload contains { _id, name, email }. These are static claims baked in at issuance -- if the user changes their name or email, the token still carries the old values until re-login.

JWT CONFIGURATION

```
const authToken = jwt.sign(
```

```
    { _id: newUser._id, name: name, email: email },
    process.env.AUTH_TOKEN
  );
```

Currently Tokens never expire. Without expiresIn, the JWT is valid indefinitely. Once issued, a token cannot be invalidated by the server. The implications:

AUTHENTICATION MIDDLEWARE

```
  const authenticateJWT = async (req, res, next) => {
    const includeAuthToken = req.body.includeAuthToken;
    const authHeader = req.headers.authorization;

    if (!authHeader && includeAuthToken !== undefined &&
includeAuthToken) {
      return res.status(401).json({ message: "No auth token provided" });
    }

    if (includeAuthToken !== undefined && !includeAuthToken) {
      next();                          // <- bypass auth entirely
    } else {
      const token = authHeader.split(" ")[1];
      const decoded = jwt.verify(token, process.env.AUTH_TOKEN);
      const user = await User.findById(decoded._id);  // <- DB call per
request
      req.user = user;
      next();
    }
  };
```

The includeAuthToken control flag is a significant design anomaly. Rather than
having separate public and protected route registrations, this system passes a

boolean flag in the request body to control whether the middleware enforces
authentication. This means:

1. Any route protected by authenticateJWT can be made effectively public by sending { includeAuthToken: false } in the body.
2. Routes like /get-product-with-id use this to serve product data to both authenticated and unauthenticated users (with different response shapes – authenticated users get isProductAlreadyInCart).
3. This is a non-standard authorization model that is easy to misuse. A developer adding a new route could accidentally pass includeAuthToken: false to a sensitive endpoint.

DB Round-Trip on Every Request: User.findById(decoded._id) is called on every authenticated request. This adds one MongoDB query to every API call. At scale, this becomes a bottleneck. The purpose is to ensure the user still exists and to load their data into req.user for use by controllers. Most controllers access req.user.cart, req.user.orders, etc., so this eager-load pattern is justified, but should eventually be replaced with a selective cache.

AUTHORIZATION (RBAC)
There is no formal RBAC. The only role distinction is isSeller: Boolean on the user document. However, this flag is never checked by the backend before allowing product uploads. The upload-product controller reads req.user._id as the seller ID and saves the product, but never verifies req.user.isSeller === true:

```
// upload-product.js -- NO seller check
const sellerId = await req.user._id;
const seller = await User.findById(sellerId);
if (!seller) return res.status(404).json({ error: "Seller not found" });
// proceeds to create product without verifying isSeller
```

Any authenticated user (including buyers) can upload products by calling /upload-product directly. The isSeller flag is checked only in the frontend (ProfileSlider renders <UploadProduct> vs <SellerAccount> based on user.isSeller). This is a trust boundary violation -- authorization logic placed in the client is security theater.

PASSWORD SECURITY

- bcrypt with saltRounds = 8. The industry baseline is 10-12. At rounds=8, bcrypt performs ~256 iterations. This is adequate but not recommended for a 2024+ deployment targeting production. Rounds=12 provides ~4096 iterations and is the current OWASP recommendation.
- Passwords are correctly never returned in API responses.
- No password strength enforcement on signup (only minimum 6-character length validation on the frontend).
- No account lockout after failed login attempts.
- No rate limiting on the login endpoint.

SESSION MANAGEMENT

There is no server-side session. Authentication is entirely token-based. The "Remember me" checkbox rendered in LoginForm.jsx is UI-only -- it has no backing implementation. Checking it does nothing.

Logout is implemented as client-side token removal:

```
localStorage.removeItem("authToken");
setUserAuthToken("");
```

The server is not notified. Token blacklisting does not exist.

GUEST USER ARCHITECTURE

A guest user account exists with hardcoded credentials in the frontend source:

```
// config.js
export const GUEST_USER_EMAIL = "guestuser@closetfashion.com";
export const GUEST_USER_PASSWORD = "guestuser";
```

This is a legitimate UX pattern for demos and portfolio showcasing, but has security implications: the credentials are public (visible in the JavaScript bundle), allowing anyone to use the guest account and potentially pollute the shared MongoDB data (adding reviews, orders, etc.). There is no data isolation or reset mechanism for the guest account.

5. DATABASE DESIGN

MONGODB ATLAS CLUSTER

The platform uses MongoDB Atlas M0 (free shared tier). This provides:
- 512 MB storage limit
- Shared cluster (multi-tenant hardware)
- No dedicated RAM
- No SLA
- No VPC peering
- Automatic backups: not included on M0

The connection string encodes credentials directly:

```
mongodb+srv://GaganSaini:gaganiscoder@cluster0.vpff1ir.mongodb.net/
  ?retryWrites=true&w=majority
```

This connection string (with plaintext credentials) is committed to the .env file in the repository. While .env is in .gitignore, the file exists on disk and is read by the deployed serverless function via Vercel environment variables. The critical risk is the credential being committed to git history if .gitignore was added after the first commit, or being leaked via error logs.

SCHEMA DESIGN

Users Collection:

```
{
  _id: ObjectId,          // MongoDB auto-generated
  userID: String,         // unused -- not set anywhere, legacy field
```

```
  name: String,
  email: String,          // no unique index, no index at all
  profileImage: String,   // base64 data URI (up to 2MB per image)
  password: String,       // bcrypt hash
  phoneNumber: String,
  address: String,        // flat string (no structured address)
  website: String,
  cart: [],               // untyped array -- contains product ID strings
  orders: [],             // untyped array -- contains { _id, quantity,
                          //   size, color, paymentMethod }
  recentsProducts: [],    // populated on signup, never written to after
  isSeller: Boolean,
  sellerEmail: String,    // set but never used (sellerEmail vs email)
  PANCardNumber: String,  // Indian tax identifier
  GSTNumber: String,      // Indian GST identifier
  TandC: Boolean,         // Terms and Conditions accepted flag
}
```

Schema Design Observations:

1. cart and orders as untyped arrays embedded in User: This is a document-embedding pattern motivated by simplicity -- fetching a user's cart or orders requires a single document read. The tradeoff is document growth: a user with hundreds of orders has an increasingly large document. MongoDB documents have a 16MB size limit, but performance degrades well before that from document bloat and the cost of loading the entire document to read or write a single array element.

2. No timestamps: Neither createdAt nor updatedAt are tracked. Historical analysis (when did user sign up, when was order placed) is impossible.

3. No email uniqueness: The Mongoose schema defines email: String with no unique: true. MongoDB will not prevent two users from registering with the same email. The login finds by User.findOne({ email }), which will arbitrarily return one of the duplicates.

4. recentsProducts is dead code: It is initialized to [] on signup and never written to. The feature was planned but not implemented.

5. userID is dead code: Set to String type but never populated. MongoDB's _id serves as the unique identifier.

6. Address is a flat string: Real e-commerce requires structured address fields (street, city, state, postal code, country) for shipping. The current address: String field cannot support address parsing, validation, or carrier API integration.

Products Collection:

```
 {
   _id: ObjectId,
   name: String,
   price: Number,
   brand: String,          // lowercased at write time
   gender: String,         // lowercased at write time
   category: String,       // lowercased at write time
   materials: String,      // lowercased at write time (single material)
   description: String,
   sizes: [],              // array of lowercase size strings: ["s","m","l"]
   colors: [],             // array of lowercase color strings
   productImages: [],      // array of base64 data URIs
   reviews: [],            // embedded subdocs: { reviewContent, starCount,
                           //   name }
   questions: [],          // embedded subdocs: { question, name }
   seller: ObjectId,       // ref: "user"
 }
```

Schema Design Observations:

1. productImages stores base64 data URIs inline. A single product with 5 images at 1MB each creates a 5MB+ MongoDB document. With a 512MB Atlas M0 limit, approximately 100 products with typical images exhaust storage. The 50MB Express body limit means a single product upload can send up to 50MB of image data in one HTTP request.

2. reviews and questions as embedded arrays without limits: A popular product could accumulate thousands of reviews. Each review is embedded in the product document, causing unbounded document growth. Loading the product detail page loads all reviews and all questions -- there is no pagination for reviews.

3. materials is a single string, not an array: Only one material per product. Mixed-material products (e.g., "60% Cotton, 40% Polyester") cannot be represented.

4. No inventory fields: No stock, stockReserved, sku, or variants fields. The product schema has no concept of quantity. Any number of users can order any product regardless of availability.

5. No createdAt / updatedAt: Product age, recency sorting, and audit trails are impossible.

6. Seller reference is an ObjectId population: seller: { type: ObjectId, ref: "User" } -- Mongoose can .populate("seller") but the controllers use User.findById(foundProduct.seller) manually instead.

INDEXING STRATEGY

No explicit indexes are defined in the Mongoose schemas. MongoDB automatically creates an index on _id. All other queries run as full collection scans:

Query	Current	Production Impact
User.findOne({ email })	Full scan	Login is O(n)
User.findById(id)	_id index	Fast
Product.find(query) with filters	Full scan	O(n) always
Product.countDocuments(query)	Full scan	Expensive at scale
Product.find({}).skip(n).limit(m)	Full scan	Skip is O(n) skip

Missing indexes that should be added immediately:
- users.email -- unique sparse index
- products.brand -- for brand filter queries
- products.category -- for category queries
- products.price -- for range queries
- products.sizes -- multikey index for array membership
- products.colors -- multikey index
- products.gender -- for gender filter
- products.seller -- for seller's product listings
- Compound: { brand: 1, category: 1, price: 1 } -- for combined filter queries

QUERY PATTERNS

Get All Products (with filters):

```
  const query = {};
  // builds: { price: { $gte, $lte }, brand: { $in }, sizes: { $in }, ... }
  const allProducts = await Product.find(query).skip(offset).limit(limit);
  const totalProducts = await Product.countDocuments(query);
```

Two sequential MongoDB queries per request: one for results, one for count. These should ideally be parallelized with:
Promise.all([Product.find(...), Product.countDocuments(...)])

The sequential pattern adds unnecessary latency.
The skip() method performs an offset scan -- MongoDB must scan and discard skip documents before returning results. This becomes expensive

as page grows. A cursor-based pagination (_id > lastSeenId) would be more efficient but requires sorted, stable result sets.

Cart Access (N+1 Pattern):

```
  for (let i = 0; i < userCart.length; i++) {
    const product = await Product.findById({ _id: userCart[i] });
    products.push({ ... });
  }
```

This is the canonical N+1 problem. For a cart with N items, N sequential MongoDB queries are made. The correct approach is:

```
// Single query with $in operator
const products = await Product.find({ _id: { $in: userCart } });
```

The fix reduces N queries to 1. The current implementation adds ~10ms per cart item from MongoDB round-trip latency alone.

6. CART SYSTEM ARCHITECTURE

CART DATA MODEL

The cart is stored as an array of product ID strings embedded in the User document:

```
 user.cart = ["64a1b2c3d4e5f6a7b8c9d0e1",
"64a1b2c3d4e5f6a7b8c9d0e2", ...]
```

Why embedded in User: Simplicity. A single User.findById loads the user and their cart in one query. No separate cart collection or join. This is appropriate at low scale and low cart size.

Design limitations:

- No quantity per item in cart. Cart can contain duplicate entries for the

same product (add to cart can be called multiple times). accessCartItems hardcodes quantity: 1 for every cart item regardless of duplicates.
- No size/color selection in cart -- these are chosen only at checkout time.
- No saved-for-later distinction.
- No cart expiry.
- No anonymous/guest cart -- cart requires login.

ADD TO CART

```
// add-to-cart.js
if (!mongoose.Types.ObjectId.isValid(req.body.productID)) {
  return res.status(400).json({ error: "Invalid product ID" });
}
await userCart.push(req.body.productID);
await req.user.save();
```

ObjectId validation is present. However, there is no check whether:
1. The product exists in the database.
2. The product is already in the cart (duplicate prevention).
3. The product is available/in stock.

The push mutates the Mongoose document array directly and saves the parent document. This is a full document write -- the entire User document is re-serialized and written to MongoDB on every add-to-cart operation.

REMOVE FROM CART

```
// remove-item.js
const products = await userCart.filter(product => product !== productID);
foundUser.cart = products;
await foundUser.save();
```

Type mismatch bug: userCart is an array of MongoDB ObjectIds (stored as BSON ObjectId type in Atlas). productID is a string from req.body.productId. The comparison product !== productID compares an ObjectId against a

string, which will never be equal. Cart removal always fails silently -- the cart array is returned unchanged (filtering with a condition that never matches), and the save writes the unmodified array back.

The correct implementation:

```
const products = userCart.filter(p => p.toString() !== productID);
```

CART ACCESS (ACCESS PATTERN ANALYSIS)

```
Frontend: CartSlider opens -> Cart.jsx mounts -> fetchCartItems()
                                        |
                                  GET /access-cart-items
                                        |
                                for each cart item:
                                  Product.findById(id) <- N queries
                                        |
                                return [{_id, name, price, ...}]
                                        |
                                setCartProducts(data.products)
                                        |
                                useEffect recalculates price summary
```

The cart is loaded fresh every time the CartSlider is opened. There is no client-side cart caching -- closing and reopening the cart triggers a full re-fetch.

PRICE CALCULATION

```
// Cart.jsx
cartProducts.forEach((product) => {
  const quantity = product.quantity;  // always 1
  totalPrice += quantity * price;
  discount += Math.trunc(quantity * price * 0.05);      // 5% discount
  deliveryCharges += price > 1000 ? 0 : Math.trunc(price * 0.01); // 1% if
< $1000
```

```
});
```

Price calculation occurs entirely in the browser. The server neither validates nor confirms pricing. The frontend sends a list of product IDs and quantities to /checkout-product, which blindly saves them without verifying price or total. This means a technically sophisticated user could manipulate the checkout payload to record orders at arbitrary prices. For a real payment integration, pricing must be computed server-side.

Note: The cart uses a 5% discount while the product detail page shows a 28% discount. These are different discount mechanisms -- the 5% is applied at cart checkout level as a loyalty discount; the 28% is a catalog-level price reduction shown as "sale price." The relationship between these two is not documented or consistent.

7. CHECKOUT & ORDER ARCHITECTURE

CHECKOUT CONTROLLER

```
// checkout-product.js
const checkoutProduct = async (req, res) => {
  const user = req.user;
  const products = req.body.products;

  user.orders = [...user.orders, ...products];
  await user.save();

  res.status(200).json({ message: "Checkout successful" });
};
```

This is the simplest possible checkout implementation. Five critical production concerns are absent:

1. No payment processing. Payment method is captured in the frontend but the paymentMethod field is included in the order object -- it is never validated, charged, or verified.

2. No inventory check. There is no stock field on products. No check is made whether the item is available.

3. No order ID generation. Each order item references the product's _id. There is no unique order identifier.

4. No transaction wrapping. If user.save() fails after products are allocated (in a system that had inventory), inventory would be leaked. Mongoose's save() is not atomic across multiple documents.

5. No cart clearing after checkout. After a successful checkout, user.cart is not emptied. The cart retains all its items even after they've been ordered. The user must manually remove items from the cart.

ORDER STATE MACHINE

There is no order state machine. Orders are placed directly into a terminal state with no lifecycle:

Actual states: ordered (implicit, only state)

ORDER HISTORY VIEW

MyOrders.jsx calls /ordered-products with the array from user.orders:

```
// get-ordered-products.js
const products = await Promise.all(
  orders.map(async (order) => {
    const product = await Product.findById(order._id);
    return { _id: product._id, name, price, productImages,
             brand, category, quantity: order.quantity };
  })
);
```

Pattern analysis: Promise.all correctly parallelizes the N individual product lookups -- this is the correct fix compared to the sequential loop in accessCartItems. However, the same $in optimization applies: Product.find({ _id: { $in: orderIds } }) would be more efficient than N parallel findById calls.

Missing order context: The history view shows current product data (fetched fresh from the products collection). If a product is deleted, updated, or repriced after an order is placed, the order history will show the new product data, not what was actually ordered. Historical order snapshots should be stored at order time, not re-fetched from the live catalog.

8. PAYMENT PROCESSING ARCHITECTURE

CURRENT IMPLEMENTATION

Payment processing is entirely simulated. The system captures payment method selection and card/UPI details in the frontend but performs no actual financial transaction. The flow:

1. User selects payment method (Cash on Delivery, UPI, Credit Card, Debit Card)
2. If UPI: user enters UPI ID
3. If Card: user enters card number, expiry date, CVC
4. User clicks "Place Order"
5. Frontend validates the form fields client-side
6. apiCaller("/checkout-product", ...) is called
7. The products payload includes paymentMethod but NOT the card/UPI details
8. Backend saves the order with paymentMethod string only

Card numbers, expiry dates, and CVCs entered by the user are never transmitted to the backend and never stored. The checkout API receives only the paymentMethod string (e.g., "creditcard").

This is actually the safest possible outcome for a mock payment system -- no sensitive financial data is transmitted or stored, avoiding PCI DSS obligations entirely. The swal message confirms this: "The item is added into your order list and delivered soon when we start delivering the products."

PAYMENT METHOD ENUMERATION

```
// config.js
export const PaymentMethods = [
  { id: "cod", name: "Cash On Delivery", value: "cod" },
  { id: "upi", name: "UPI", value: "upi" },
  { id: "credit-card", name: "Credit Card", value: "creditcard" },
  { id: "debit-card", name: "Debit Card", value: "debitcard" },
];
```

UPI is an Indian payment rail (Unified Payments Interface), confirming the platform targets the Indian market. PAN Card and GST number validation in the seller form also confirms India as the primary market.

PAYMENT SECURITY VALIDATION (FRONTEND)

The buyProductPageUserValidations includes regex validation for:
- UPI ID: /^[a-zA-Z0-9.\-_]{2,256}@[a-zA-Z]{2,64}$/ -- valid UPI VPA format
- Card Number: /^[0-9]{13,19}$/ -- standard 13-19 digit card number
- Expiry Date: /^(0[1-9]|1[0-2])\/?([0-9]{4}|[0-9]{2})$/ -- MM/YY or MM/YYYY
- CVC: /^[0-9]{3,4}$/ -- 3-4 digit CVC

These validations exist only for UX completeness. They prevent malformed inputs from being submitted but since the data is never sent to the backend, the validation serves only to provide a realistic-looking form experience.

PRODUCTION PAYMENT INTEGRATION ROADMAP

To implement real payments for the Indian market, the recommended integration is Razorpay (dominant Indian payment gateway) or PayU India. The integration would require:

1. Backend: Create order via Razorpay Orders API, receive order_id
2. Frontend: Initialize Razorpay checkout with order_id, collect payment via hosted checkout
3. Backend webhook: Verify payment signature via razorpay.webhooks.verify(), update order status
4. Idempotency: Webhook handlers must be idempotent -- Razorpay retries webhooks on failure
5. PCI DSS: All card data handled by Razorpay's hosted fields; platform never touches card numbers

9. PRODUCT CATALOG & SEARCH

PRODUCT LISTING ARCHITECTURE

The product listing is driven by a single /get-all-products endpoint that handles filtering, search, and pagination simultaneously. The query builder:

```
let query = {};
// Price range filter
query.price = { $gte: min, $lte: max };
// Brand filter (OR)
if (brands.length > 0) query.brand = { $in: brands };
// Size filter (array membership)
if (sizes.length > 0) query.sizes = { $in: sizes };
// Gender filter
if (genders.length > 0) query.gender = { $in: genders };
// Material filter
if (materials.length > 0) query.materials = { $in: materials };
// Color filter
if (colors.length > 0) query.colors = { $in: colors };
```

```
// Search (regex across name, description, brand)
if (searchQuery) {
  query.$or = [
    { name: { $regex: searchQuery, $options: "i" } },
    { description: { $regex: searchQuery, $options: "i" } },
    { brand: { $regex: searchQuery, $options: "i" } },
  ];
}
```

All filter dimensions are combined with implicit AND. There is no OR between filter categories (e.g., cannot search for "Nike OR Adidas shoes"). This is standard e-commerce filter behavior.

PAGINATION STRATEGY

```
// Pagination is DISABLED when filters are active
const offset = filterExist ? 0 : (page - 1) * limit;
const allProducts = await Product.find(query).skip(offset).limit(limit);
```

This is a significant behavioral choice: when the user applies any filter, all matching products are returned in a single response (up to the MongoDB result limit). The page counter is ignored. This means:
- Browsing without filters: paginated (12 items/page, infinite scroll)
- Browsing with any filter: all results returned at once

The rationale is likely simplicity -- filter results may be small enough that pagination is unnecessary. But as the catalog grows, a filter for "Brand: Nike" could return thousands of products in one response.

SEARCH IMPLEMENTATION

The implemented search (via $regex in getAllProducts) works correctly for the current catalog size. The /search-product endpoint is a broken dead route – it references product.company which is not a field on the Product

schema. Every call to this endpoint throws a JavaScript runtime error inside the filter() callback. It is registered in server.js at line 78 with a comment "// Not Working".

The working search path is via the searchQuery parameter in /get-all-products, which integrates search with filtering. The frontend uses SearchContext to propagate the search term from the Navbar to the Home component, which includes it in the debounced API call.

Search behavior on the frontend:

```
// Home.jsx -- filter reset when search is active
useEffect(() => {
  if (filtersActive || searchQuery?.length > 0) {
    setAllProducts([...data?.products]);  // replace, not append
  } else {
    setAllProducts(prev => [...prev, ...data?.products]);  // append
  }
}, [data?.products]);
```

The 250ms debounce on the search means the API is called 250ms after the user stops typing. The useDebouncedAPI hook correctly uses useRef for timer management.

FILTER ARCHITECTURE

```
AllFilters = [
  { id: "brands", options: Brands },       // 9 brands
  { id: "sizes", options: Sizes },         // 5 sizes: S M L XL XXL
  { id: "genders", options: Genders },     // Men, Women, Unisex
  { id: "materials", options: Materials }, // 7 materials
]
```

Filter state is maintained in Home.jsx as a single object. Each filter toggle calls setSelectedFilters to immutably add/remove values from the relevant array. The Filters component receives this state as props and renders checkboxes accordingly.

Price range behavior: The RangePicker component calls setSelectedFilters inside a useEffect triggered on every slider change. This means every slider movement (including intermediate positions while dragging) triggers a debounced API call. The 250ms debounce on the API hook absorbs rapid slider movement.

10. INVENTORY DESIGN

CURRENT STATE: NO INVENTORY SYSTEM

The platform has no inventory management whatsoever. The Product schema has no stock-related fields:

- No stock: Number (quantity available)
- No stockReserved: Number (items in active carts/orders)
- No sku: String (stock keeping unit)
- No variants (size x color stock matrix)

Every product is implicitly treated as having infinite stock. Any number of users can order the same product. The product detail page shows "In Order" (green) regardless of actual availability.

IMPLICATIONS

1. Overselling: The system will happily accept orders for products that have zero physical stock.
2. No reservation: There is no cart-level stock reservation to prevent the classic race condition where 100 users add the last unit to their cart simultaneously.
3. No flash sale protection: High-demand drops would simply record thousands of orders regardless of supply.

INVENTORY ARCHITECTURE REQUIRED

For production, inventory requires:

```
Product Variant
+-- productId: ObjectId
+-- size: String
+-- color: String
```

+-- sku: String
+-- stockTotal: Number
+-- stockReserved: Number <- items currently in carts
+-- stockAvailable: Number <- stockTotal - stockReserved
+-- version: Number <- optimistic concurrency

Cart Reservation (TTL-based)
+-- cartId: String
+-- productVariantId: ObjectId
+-- quantity: Number
+-- expiresAt: Date <- TTL index, auto-expires after 30min

The stock deduction flow with optimistic concurrency:

User adds to cart:

1. findOneAndUpdate(
   { _id: variantId, stockAvailable: { $gte: quantity } },
   { $inc: { stockAvailable: -quantity, stockReserved: +quantity } },
   { new: true })
2. If result is null -> out of stock
3. Create CartReservation with TTL
4. If cart expires before checkout -> reverse reservation

User checks out:

1. Verify CartReservation still valid
2. findOneAndUpdate to decrement stockReserved and stockTotal
3. Create Order document
4. Delete CartReservation

—

END OF THE TRAINING REPORT